

NAME : Syed Salman .S

Reg No:192524076

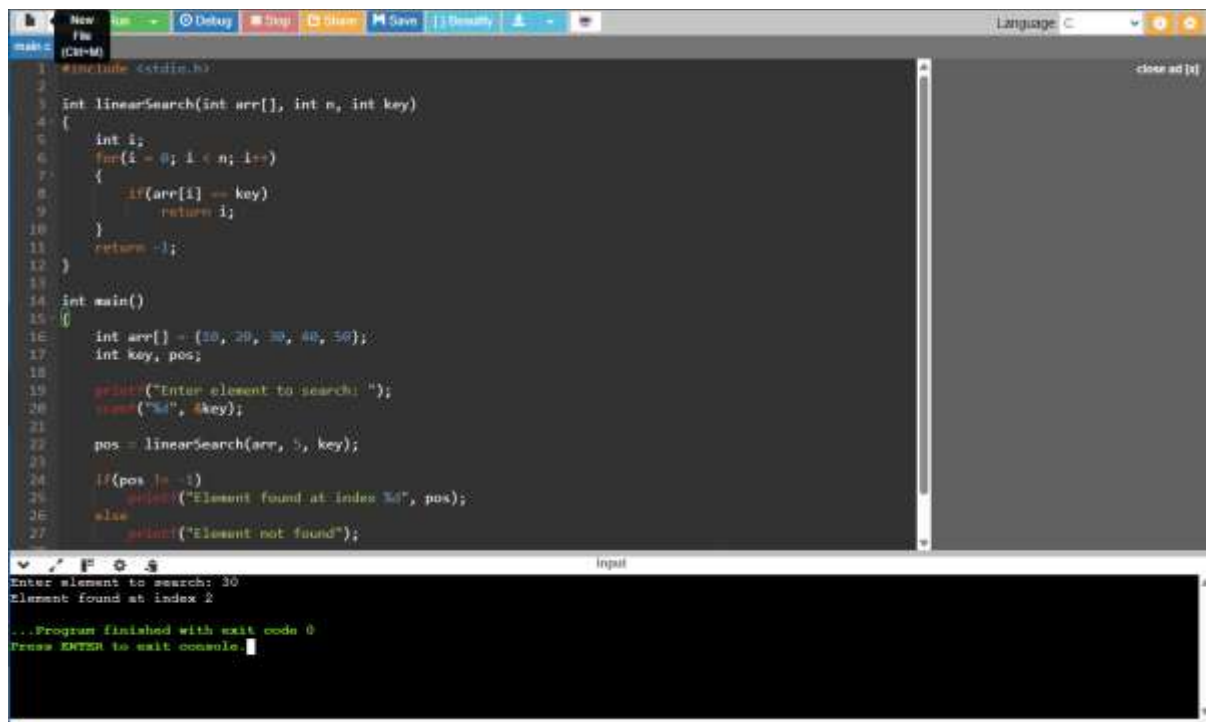
Subject Name :Data Structures

Subject code:0316

ASSESSMENT TOLL-C02(AT-2)

1. Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
    for(int i = 0; i &lt;= n; i++) {  
        if(arr[i] == key)  
            return i;  
    }  
    return -1;  
}
```



The screenshot shows a C++ IDE with a file named 'main.cpp'. The code implements a linear search function and a main function. The linear search function has a loop that iterates from `i = 0` to `i <= n`, which is a bug because it should be `i < n`. The main function initializes an array `arr` with values {10, 20, 30, 40, 50}, prompts the user to enter an element to search, and calls the `linearSearch` function. The output shows that the element 30 was found at index 2.

```
1 #include <stdio.h>  
2  
3 int linearSearch(int arr[], int n, int key)  
4 {  
5     int i;  
6     for(i = 0; i <= n; i++)  
7     {  
8         if(arr[i] == key)  
9             return i;  
10    }  
11    return -1;  
12 }  
13  
14 int main()  
15 {  
16     int arr[] = {10, 20, 30, 40, 50};  
17     int key, pos;  
18  
19     printf("Enter element to search: ");  
20     scanf("%d", &key);  
21  
22     pos = linearSearch(arr, 5, key);  
23  
24     if(pos != -1)  
25         printf("Element found at index %d", pos);  
26     else  
27         printf("Element not found");  
28 }
```

Enter element to search: 30
Element found at index 2
...Program finished with exit code 0
Press ENTER to wait console.

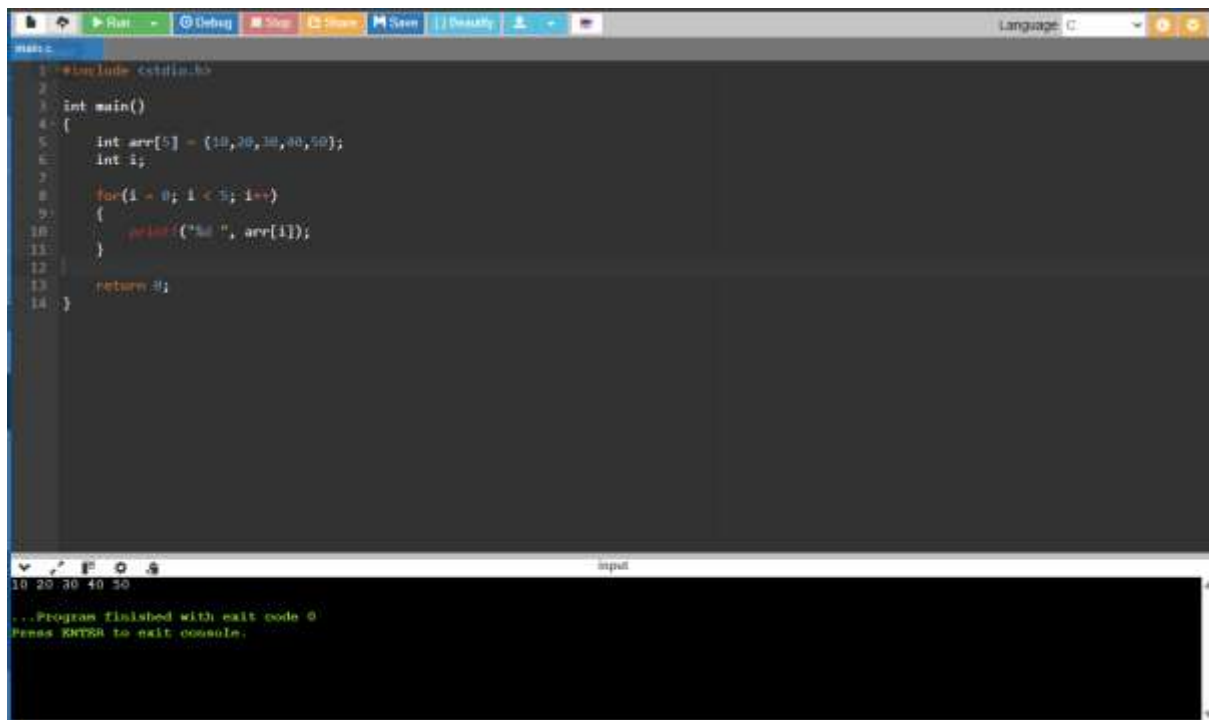
2. Buggy Code in an array

```
#include <stdio.h>

int main() {
    int arr[5] = {10,20,30,40,50};
    for(int i=0;i<=5;i++)
        printf("&quot;%d &quot;",arr[i]);
    return 0;
}
```

Expected output:

10 0 30 40 50



The screenshot shows a C program being executed in a debugger. The code defines an array `arr` of size 5 with values {10, 20, 30, 40, 50} and iterates from `i=0` to `i=5`. The debugger's console window shows the output "10 20 30 40 50" and a message "...Program finished with exit code 0". The program's memory dump shows the array contents correctly. The console also displays "Press ENTER to exit console."

3. Identify an errors in the following snippet

```
int binarySearch(int arr[], int n, int key)
```

```
{
```

```
int low = 0, high = n - 1;
```

```
while(low &lt; high)
```

```
{ int mid = (low + high) / 2;
```

```
if(arr[mid] == key) return mid;
```

```
else if(arr[mid] &lt; key)
```

```
low = mid;
```

```
else high = mid;
```

```
}
```

```
return -1;
```

```
}
```



The screenshot shows a C++ IDE with a dark theme. The code editor displays a binary search function and a main function. The function signature is `int binarySearch(int arr[], int n, int key)`. The code includes `<stdio.h>`. The `while` loop condition is `low < high`. The `if` statement checks `arr[mid] == key`. The `else if` statement checks `arr[mid] < key`. The `else` statement checks `arr[mid] > key`. The `return` statement returns `-1`. The `main` function declares `int arr[] = {10, 20, 30, 40, 50};` and `int key, pos;`. The output window shows the input `30` and the message `Element found at index 2`. The program finished with exit code 0.

```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int n, int key)
4 {
5     int low = 0;
6     int high = n - 1;
7
8     while(low < high)
9     {
10         int mid = (low + high) / 2;
11
12         if(arr[mid] == key)
13             return mid;
14         else if(arr[mid] < key)
15             low = mid + 1;
16         else
17             high = mid - 1;
18     }
19
20     return -1;
21 }
22
23 int main()
24 {
25     int arr[] = {10, 20, 30, 40, 50};
26     int key, pos;
27
28     printf("Enter element: ");
29     scanf("%d", &key);
30
31     pos = binarySearch(arr, 5, key);
32
33     if(pos != -1)
34         printf("Element found at index %d\n", pos);
35     else
36         printf("Element not found\n");
37
38     return 0;
39 }
```

Enter element: 30
Element found at index 2
...Program finished with exit code 0
Press ENTER to exit console.

4. Buggy Code in Stack Push Operation

```
#include <stdio.h>

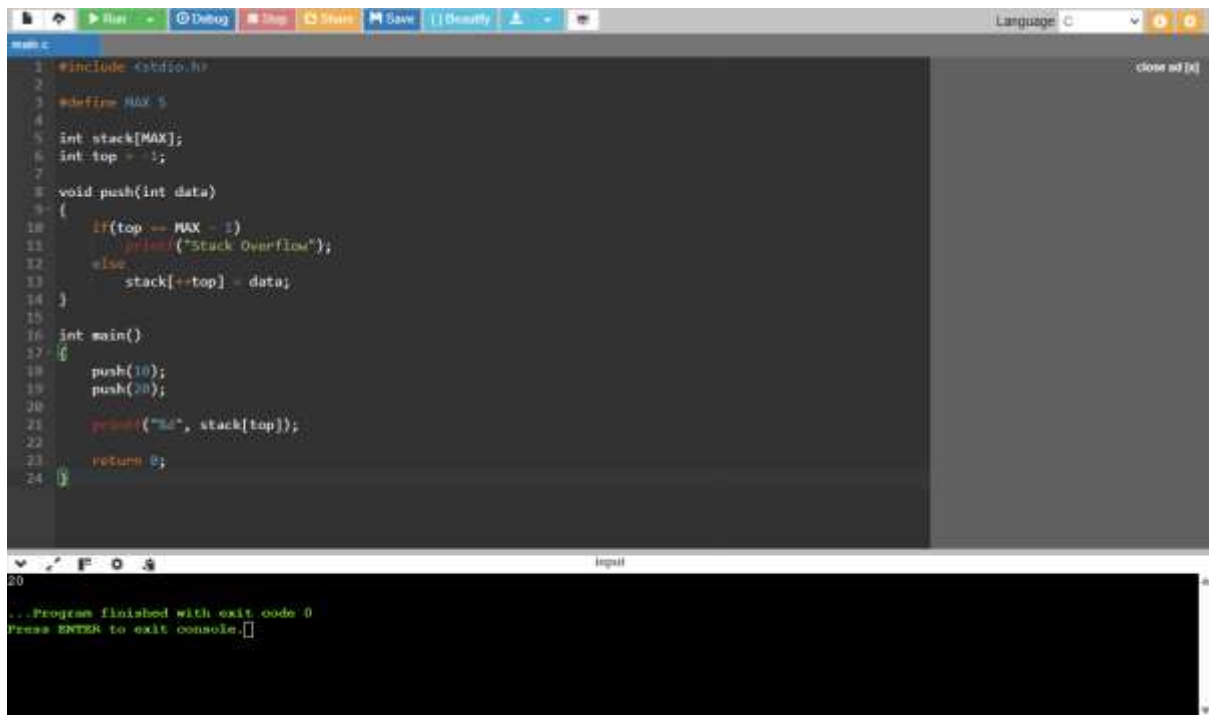
#define MAX 5

int stack[MAX];

int top = -1;

void push(int data) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[top++] = data;
}

int main() {
    push(10);
    push(20);
    printf("%d\n", stack[top]);
}
```



```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 void push(int data)
9 {
10     if(top == MAX - 1)
11         printf("Stack Overflow\n");
12     else
13         stack[++top] = data;
14 }
15
16 int main()
17 {
18     push(10);
19     push(20);
20     printf("%d\n", stack[top]);
21     return 0;
22 }
```

...Program finished with exit code 0
Press ENTER to exit console.

5. Buggy Code in Stack POP Operation

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = 2;
```

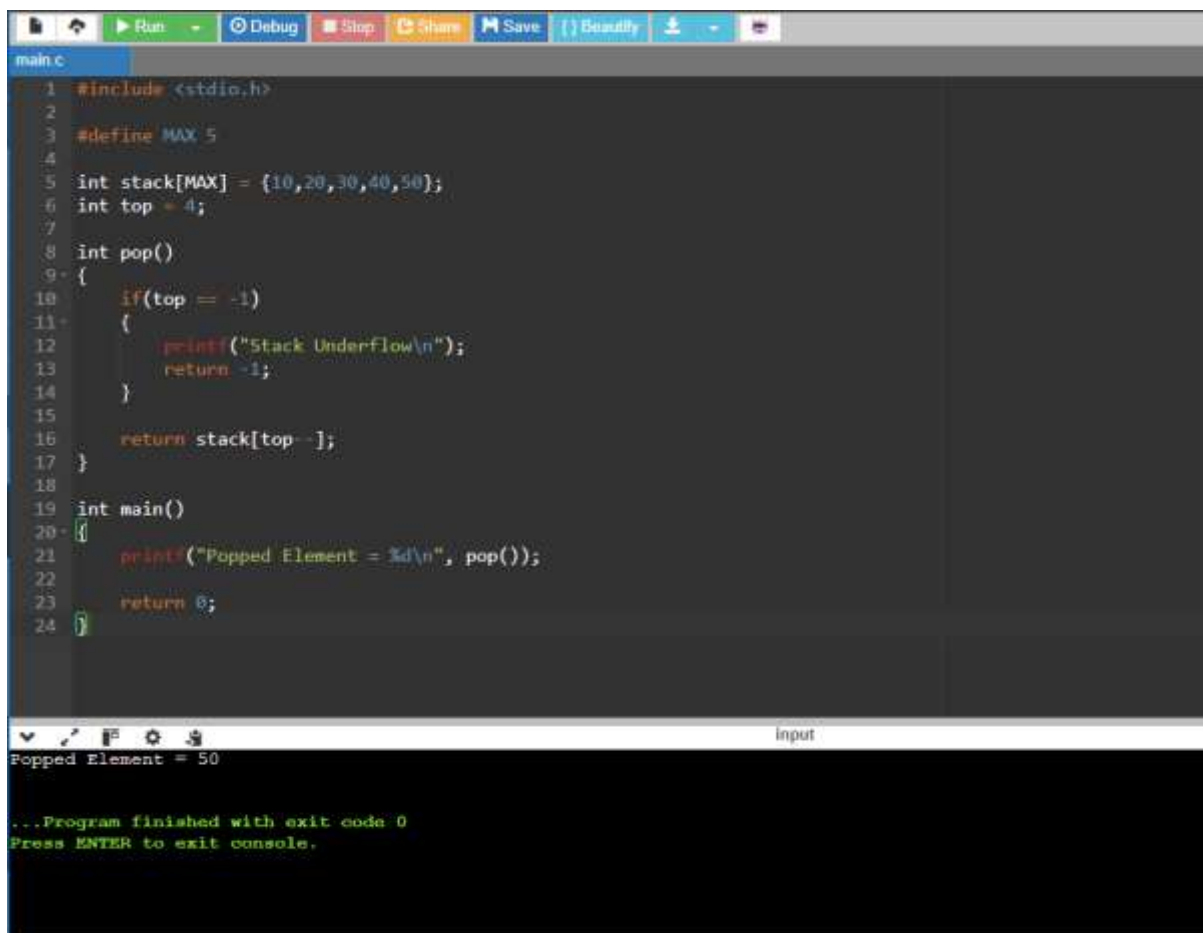
```
int pop() {
```

```
if (top == -1)
```

```
return -1;
```

```
return stack[++top];
```

```
}
```



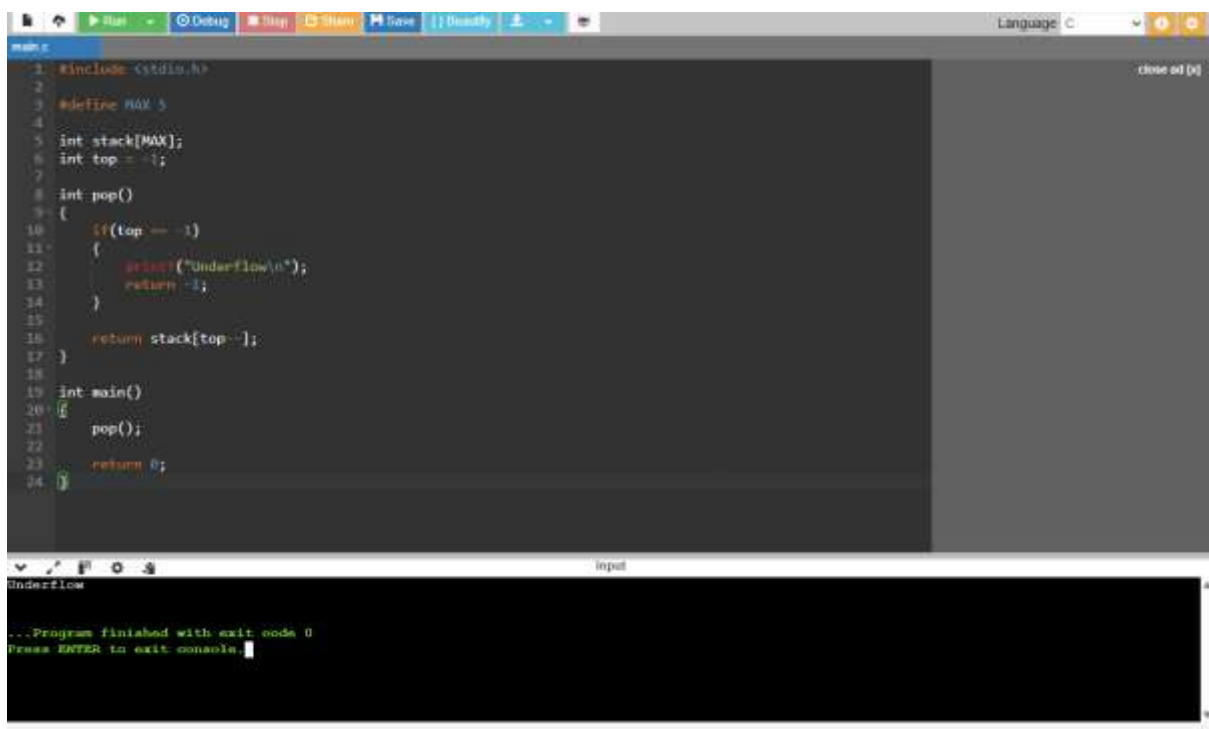
```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX] = {10, 20, 30, 40, 50};
6 int top = 4;
7
8 int pop()
9 {
10     if (top == -1)
11     {
12         printf("Stack Underflow\n");
13         return -1;
14     }
15     return stack[top-1];
16 }
17
18 int main()
19 {
20     printf("Popped Element = %d\n", pop());
21     return 0;
22 }
```

Popped Element = 50

...Program finished with exit code 0
Press ENTER to exit console.

6. Buggy Code in underflow operation

```
int pop() {  
    if(top == 0) {  
        printf("&quot;Underflow&quot;);  
        return -1;  
    }  
    return stack[top--];  
}
```



The screenshot shows a C++ IDE with a code editor and a console window. The code defines a stack with a maximum size of 5 and a top pointer initialized to -1. The pop() function has a bug: it checks if top is 0 instead of -1. When the stack is empty (top = -1), it prints "Underflow" and returns -1. In the main function, pop() is called once, which triggers the underflow condition.

```
1. #include <stdio.h>  
2.  
3. #define MAX 5  
4.  
5. int stack[MAX];  
6. int top = -1;  
7.  
8. int pop()  
9. {  
10.     if(top == 0)  
11.     {  
12.         printf("Underflow\n");  
13.         return -1;  
14.     }  
15.  
16.     return stack[top--];  
17. }  
18.  
19. int main()  
20. {  
21.     pop();  
22.  
23.     return 0;  
24. }
```

Underflow

...Program finished with exit code 0
Press ENTER to exit console.

7. Find the issue in the following code snippet.

```

#include<stdio.h>

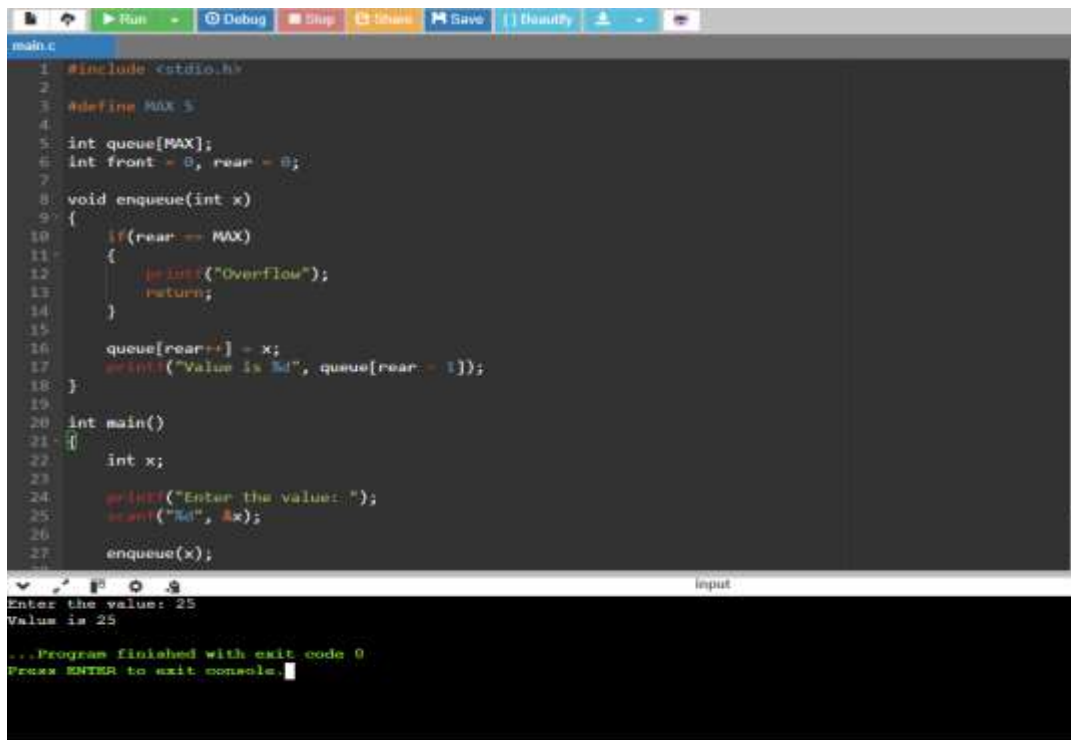
#define MAX 5

void main()
{
    int queue[MAX],x;
    int front = 0, rear = 0;

    printf(&quot;Enter the value&quot;);
    scanf(&quot;%d&quot;,&amp;x);

    void enqueue(int x) {
        if(rear == MAX) {
            printf(&quot;Overflow&quot;);
            return;
        }
        else{
            queue[rear++] = x;
            printf(&quot;Value is %d&quot;,, queue[rear]);
        }
    }
}

```



```

main.c
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = 0, rear = 0;
7
8 void enqueue(int x)
9 {
10     if(rear == MAX)
11     {
12         printf("Overflow");
13         return;
14     }
15     queue[rear++] = x;
16     printf("Value is %d", queue[rear - 1]);
17 }
18
19
20 int main()
21 {
22     int x;
23
24     printf("Enter the value: ");
25     scanf("%d", &x);
26
27     enqueue(x);
28 }

```

Enter the value: 25
Value is 25
...Program finished with exit code 0
Press ENTER to exit console.

8. Buggy code in node creation

```

#include <stdio.h>

```

```

#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *newNode;
    newNode->data = 10;
    newNode->next = NULL;

    printf("&quot;%d&quot;, newNode->data);
    return 0;
}

```

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 int main()
11 {
12     struct Node *newNode;
13
14     newNode = (struct Node *)malloc(sizeof(struct Node));
15
16     newNode->data = 10;
17     newNode->next = NULL;
18
19     printf("%d", newNode->data);
20
21     free(newNode);
22
23     return 0;
24 }

```

...Program finished with exit code 0
Press ENTER to exit console.

9. Include missing statements and display the list of elements in linkedlist

```

void display(struct Node *head)
{

```



```

while(head != NULL);
{
printf("&quot;%d &quot;, head-&gt;data);
head = head-&gt;next;
} }

```

The screenshot shows a C code editor with a file named 'main.c'. The code implements a linked list with a 'display' function and a 'main' function. The 'display' function uses a 'while' loop to traverse the list. The 'main' function initializes three nodes, sets the 'head' pointer to the first node, and calls the 'display' function. The console output at the bottom shows the program finished with exit code 0.

```

main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 void display(struct Node *head)
11 {
12     while(head != NULL)
13     {
14         printf("%d ", head->data);
15         head = head->next;
16     }
17 }
18
19 int main()
20 {
21     struct Node *head, *second, *third;
22
23     head = (struct Node *)malloc(sizeof(struct Node));
24     second = (struct Node *)malloc(sizeof(struct Node));
25     third = (struct Node *)malloc(sizeof(struct Node));
26
27     head->data = 10;
28     head->next = second;
29
30
...Program finished with exit code 0
Press ENTER to exit console.

```

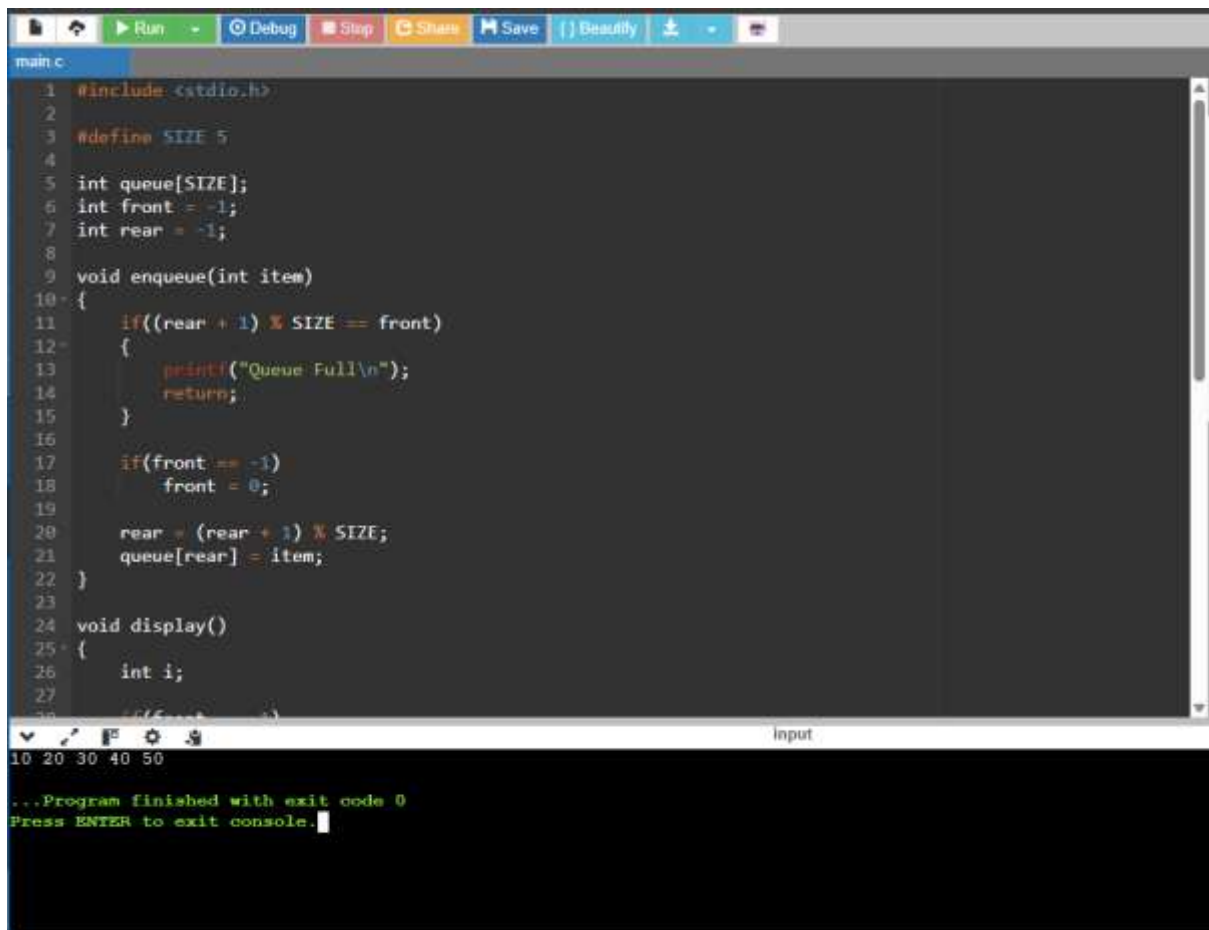
10. Bug in Queue Full Condition-Identify and fix the errors.

```

#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

```

```
void enqueue(int item)
{
if(rear == SIZE-1)
{
printf("&quot;Queue Full&quot;");
return;
}
if(front == -1)
front = 0;
rear = (rear + 1) % SIZE;
```



The screenshot shows a C++ IDE with a dark theme. The editor window displays the implementation of a queue. The code includes `<stdio.h>`, defines `SIZE` as 5, and declares an array `queue` of size 5. It initializes `front` to -1 and `rear` to -1. The `enqueue` function checks if the queue is full by comparing `(rear + 1) % SIZE` with `front`. If full, it prints "Queue Full\n" and returns. If `front` is -1, it sets `front` to 0. Then, it increments `rear` and stores the `item` at `queue[rear]`. The `display` function is also declared. The console window at the bottom shows the program finished with exit code 0 and prompts the user to press ENTER to exit the console.

```
main.c
1 #include <stdio.h>
2
3 #define SIZE 5
4
5 int queue[SIZE];
6 int front = -1;
7 int rear = -1;
8
9 void enqueue(int item)
10 {
11     if((rear + 1) % SIZE == front)
12     {
13         printf("Queue Full\n");
14         return;
15     }
16
17     if(front == -1)
18         front = 0;
19
20     rear = (rear + 1) % SIZE;
21     queue[rear] = item;
22 }
23
24 void display()
25 {
26     int i;
27
28     //if(front == -1)
29     //    return;
30
31     for(i = front; i < rear; i++)
32         printf("%d ", queue[i]);
33     printf("\n");
34 }
35
36 int main()
37 {
38     int n;
39     printf("Enter number of elements: ");
40     scanf("%d", &n);
41
42     while(n > 0)
43     {
44         int item;
45         printf("Enter element: ");
46         scanf("%d", &item);
47         enqueue(item);
48         n--;
49     }
50
51     display();
52
53     return 0;
54 }
```

10 20 30 40 50

...Program finished with exit code 0
Press ENTER to exit console.